

MATH00041: Introduction to Programming

NumPy I

Lecture 8

Archontis Giannakidis, PhD

Outline

NumPy Library

NumPy is a Python library that allows programmers to work with arrays and matrices. Arrays are a way to store and manipulate data in a structured way. They are similar to lists or tables but have some additional features that make them more powerful and efficient.

For example, arrays can perform mathematical operations on entire sets of data at once, which can be faster and more convenient than doing the same operations on each item in a list individually. NumPy is designed specifically for working with these arrays, providing many functions and tools for manipulating and analysing them.

NumPy is a valuable tool for programmers working with large data sets or performing complex mathematical operations on arrays. By using NumPy, they can write more efficient and effective code that can easily handle these tasks.

Why Use NumPy?

Python has lists that serve a similar purpose to arrays, but they are relatively slow to process. On the other hand, NumPy aims to provide an array object up to 50 times faster than traditional Python lists. The array object in NumPy is called `ndarray`, and it comes equipped with a variety of supporting functions that make working with it extremely easy. Arrays are widely used in data science, where speed and resources are crucial factors to consider.

Why is NumPy Faster Than Lists?

NumPy arrays are stored in a contiguous block of memory, which means they can be accessed and manipulated efficiently by various processes. This phenomenon is referred to as "locality of reference" in computer science. This is the primary reason why NumPy performs better than lists and is optimised to work with the latest CPU architectures.

Installation of NumPy

If you have Python and PIP already installed on a system, then installation of NumPy is very easy.

Install it using this command:

```
C:\Users\Your Name>pip install numpy
```

If this command fails, then use a Python distribution that already has NumPy installed like Anaconda, Spyder etc.

Import NumPy

After installing NumPy, you can import it in your applications using the 'import' keyword:

```
import numpy
```

Now NumPy is imported and ready to use.

Example:

```
import numpy
```

```
arr = numpy.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

```
>>[1 2 3 4 5]
```

NumPy as np

NumPy is usually imported under the `np` alias: `import numpy as np`

The NumPy package can now be referred to as `np` instead of `numpy`.

Example:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

```
>>[1 2 3 4 5]
```

Checking NumPy Version

The version string is stored under `__version__` attribute.

Example:

```
import numpy as np
```

```
print(np.__version__)
```

```
>>1.16.3
```

Create a NumPy ndarray Object

NumPy is a popular library in Python used for scientific computing. Its array object, called ndarray, provides a powerful way to work with arrays. An array is a collection of elements of the same data type, such as integers or floating-point numbers, arranged in a grid of rows and columns. With NumPy's ndarray, you can perform operations on arrays such as arithmetic, slicing, indexing, and more. To create an ndarray, you can use the `array()` function in NumPy.

Example:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

print(arr)

print(type(arr))
```

```
>>[1 2 3 4 5]
<class 'numpy.ndarray'>
```

To create an **ndarray**, we can pass a list, tuple or any array-like object into the **array()** method, and it will be converted into an **ndarray**:

Example:

```
import numpy as np

arr = np.array((1, 2, 3, 4, 5))

print(arr)
```

```
>>[1 2 3 4 5]
```

Dimensions in Arrays

Arrays consist of one or more levels of nested arrays which are referred to as dimensions. Essentially, a dimension in an array is a single level of array depth.

❖ **nested array**: are arrays that have arrays as their elements.

0-D Arrays

0-D arrays, or Scalars, are the elements in an array. Each value in an array is a 0-D array.

Example:

Create a 0-D array with value 42

```
import numpy as np
```

```
arr = np.array(42)
```

```
print(arr)
```

```
>>42
```

1-D Arrays

An array with 0-D arrays as elements is a 1-D array, the most basic type.

Example:

Create a 1-D array containing the values 1,2,3,4,5:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5])
```

```
print(arr)
```

```
>>[1 2 3 4 5]
```

2-D Arrays

An array containing 1-D arrays is a 2-D array commonly used to represent matrices or 2nd-order tensors. NumPy has a whole sub-module dedicated towards matrix operations called **numpy.mat**

Example:

Create a 2-D array containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr)
```

```
>>[[1 2 3]
     [4 5 6]]
```

3-D arrays

An array containing 2-D arrays (matrices) is a 3-D array commonly used to represent a 3rd order tensor.

Example:

Create a 3-D array with two 2-D arrays, both containing two arrays with the values 1,2,3 and 4,5,6:

```
import numpy as np

arr = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])

print(arr)
```

```
>>[[[1 2 3]
      [4 5 6]]

     [[1 2 3]
      [4 5 6]]]
```

np.zeros

The **zeros** function creates an array containing any number of zeros:

Example:

Create a array of zeros:

```
import numpy as np
```

```
arr = np.zeros(5)
```

```
print(arr)
```

```
>>array([0., 0., 0., 0., 0.])
```

np.ones

The **ones** function creates an array containing any number of ones:

Example:

Create a array of ones:

```
import numpy as np
```

```
arr = np.ones(5)
```

```
print(arr)
```

```
>>array([1., 1., 1., 1., 1.])
```

np.full

Creates an array of the given shape initialised with the given value.

Example:

Create a 3x4 matrix full of π :

```
import numpy as np

arr = np.full((3,4), np.pi)

print(arr)
```

```
>>array([[3.14159265, 3.14159265, 3.14159265, 3.14159265],
        [3.14159265, 3.14159265, 3.14159265, 3.14159265],
        [3.14159265, 3.14159265, 3.14159265, 3.14159265]])
```

np.arange

You can create an `ndarray` using NumPy's `arange` function, which is similar to python's built-in `range` function:

Example:

Create an array with integer and float using arange function:

```
import numpy as np
```

```
arr1 = np.arange(1, 5)
```

```
arr2 = np.arange(1.0, 5.0)
```

```
print(arr1)
```

```
print(arr2)
```

```
>>array([1, 2, 3, 4])
```

```
>>array([1., 2., 3., 4.])
```

np.linspace

When working with floating point numbers, it is typically more advantageous to use the `linspace` function instead of `arange`. The `linspace` function generates an array that includes a specified number of points that are evenly distributed between two values (keep in mind that the upper limit value is included, unlike in the case of `arange`).

Example:

Create an array with `linspace` function:

```
import numpy as np
```

```
arr = np.linspace(0, 5/3, 6)
```

```
print(arr)
```

```
>>[0.          0.33333333 0.66666667 1.          1.33333333 1.66666667]
```

Check The Number of Dimensions

NumPy Arrays provide the `ndim` attribute, which returns an integer indicating the number of dimensions in the array.

Example:

Check how many dimensions the arrays have:

```
import numpy as np
```

```
a = np.array(42)
```

```
b = np.array([1, 2, 3, 4, 5])
```

```
c = np.array([[1, 2, 3], [4, 5, 6]])
```

```
d = np.array([[[1, 2, 3], [4, 5, 6]], [[1, 2, 3], [4, 5, 6]]])
```

```
print(a.ndim)
```

```
print(b.ndim)
```

```
print(c.ndim)
```

```
print(d.ndim)
```

```
>>0
```

```
1
```

```
2
```

```
3
```

Higher Dimensional Arrays

An array can have any number of dimensions. When creating an array, the number of dimensions can be defined using the `ndmin` argument.

Example:

Create an array with 5 dimensions and verify that it has 5 dimensions:

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)
print('number of dimensions :', arr.ndim)
```

```
>>[[[[[1 2 3 4]]]]]
number of dimensions : 5
```

Access Array Elements

When you index an array, you are essentially accessing a specific element within that array. The element can be accessed by referencing its corresponding index number. In NumPy arrays, index numbers start at 0, meaning that the first element of the array has an index 0, the second element has an index 1, and so on.

Example:

Get the first element from the following array:

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr[0])
```

```
>>1
```

Access 2-D Arrays

To retrieve elements from 2-D arrays, we can use comma-separated integers that represent the element's dimension and index. 2-D arrays can be thought of as a table with rows and columns, where the dimension is equivalent to the row and the index is equivalent to the column.

Example:

Access the element on the first row, second column:

```
import numpy as np
```

```
arr = np.array([[1,2,3,4,5], [6,7,8,9,10]])
```

```
print('2nd element on 1st row: ', arr[0, 1])
```

```
>>2nd element on 1st dim:  2
```

Slicing arrays

In Python, "slicing" refers to the process of extracting specific elements from a sequence, based on their position. To slice a sequence, we use square brackets and specify the starting and ending indices of the slice separated by a colon. For example, `[start:end]`. Additionally, we can define the step size of the slice, like this: `[start:end:step]`.

If we don't specify the "start" index, Python assumes that we want to start from the beginning of the sequence. If we don't specify the "end" index, Python assumes that we want to slice until the end of the sequence. If we don't specify the "step" size, Python assumes that we want to extract every element with a step size of 1.

Example:

Slice elements from index 1 to index 5 from the following array:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])
print(arr[1:5])
```

```
>>[2 3 4 5]
```

❖ **Note:** The result includes the start index but excludes the end index.

Example:

Slice elements from index 4 to the end of the array:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[4:])

>>[5 6 7]
```

Example:

Slice elements from the beginning to index 4 (not included):

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[:4])

>>[1 2 3 4]
```

Negative Slicing

Use the minus operator to refer to an index from the end:

Example:

Slice from the index 3 from the end to index 1 from the end:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
```

```
print(arr[-3:-1])
```

```
>>[5 6]
```

STEP

Use the **step** value to determine the step of the slicing:

Example:

Return every other element from index 1 to index 5:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7])

print(arr[1:5:2])
```

```
>>[2 4]
```

Slicing 2-D Arrays

Example:

From the second element, slice elements from index 1 to index 4 (not included):

```
import numpy as np
```

```
arr = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
```

```
print(arr[1, 1:4])
```

```
>>[7 8 9]
```

❖ **Note:** Remember that the second element has index 1.

Data Types in Python

By default, Python have these data types:

- 1) **strings** - used to represent text data, the text is given under quote marks. e.g. "ABCD"
- 2) **integer** - used to represent integer numbers. e.g. -1, -2, -3
- 3) **float** - used to represent real numbers. e.g. 1.2, 42.42
- 4) **boolean** - used to represent True or False.
- 5) **complex** - used to represent complex numbers. e.g. $1.0 + 2.0j$, $1.5 + 2.5j$

Data Types in NumPy

NumPy has some extra data types and refers to data types with one character, like **i** for integers, **u** for unsigned integers etc.

Below is a list of all data types in NumPy and the characters used to represent them.

1. **i** - integer
2. **b** - boolean
3. **u** - unsigned integer
4. **f** - float
5. **c** - complex float
6. **m** - timedelta
7. **M** - datetime]
8. **O** - object
9. **S** - string
10. **U** - unicode string
11. **V** - a fixed chunk of memory for other type (void)

Checking the Data Type of an Array

The NumPy array object has a property called `dtype` that returns the data type of the array:

Example:

Get the data type of an array object:

```
import numpy as np

arr = np.array([1, 2, 3, 4])

print(arr.dtype)

>>int64
```

Example:

Get the data type of an array object:

```
import numpy as np

arr = np.array(['apple', 'banana', 'cherry'])

print(arr.dtype)

>><U6
```

Creating Arrays with a Defined Data Type

In Python, an array is a data structure that can store a collection of elements of the same data type. We use the `array()` function to create an array, and this function can take an optional argument `'dtype'`. With this argument, we can specify the expected data type of the array elements. For instance, if we want to create an array of integers, we can set the `'dtype'` argument to `'int'`. This way, Python will ensure that all the elements in the array are of integer type.

Example:

Create an array with data type string:

```
import numpy as np

arr = np.array([1, 2, 3, 4], dtype='S')

print(arr)
print(arr.dtype)
```

```
>>[b'1' b'2' b'3' b'4']
|S1
```

For **i**, **u**, **f**, **S** and **U** we can define size as well.

Example:

Create an array with data type 4 bytes integer:

```
import numpy as np

arr = np.array([1, 2, 3, 4], dtype='i4')

print(arr)
print(arr.dtype)
```

```
>>[1 2 3 4]
int32
```

Converting Data Type on Existing Arrays

To change the data type of an existing array, the best approach is to create a copy of the array using the `astype()` method.

The `astype()` method generates a copy of the array while allowing you to specify the data type as a parameter.

You can specify the data type using a string, such as `'f'` for float, `'i'` for integer, or you can directly use `float` for float and `int` for integer.

Example:

Change data type from float to integer by using `int` as parameter value:

```
import numpy as np
```

```
arr = np.array([1.1, 2.1, 3.1])
```

```
newarr = arr.astype(int)
```

```
print(newarr)
```

```
print(newarr.dtype)
```

```
>>[1 2 3]
```

```
int32
```

Example:

Change data type from integer to boolean:

```
import numpy as np

arr = np.array([1, 0, 3])

newarr = arr.astype(bool)

print(newarr)
print(newarr.dtype)
```

```
>> [ True False  True]
bool
```

NumPy Array Copy vs View

The Difference Between Copy and View

There are two ways to create a new array from an existing array, by copying or by creating a view. The main difference between the two is that a copy creates a new array with its own data, while a view only provides a new way to look at the original array.

When you make changes to a copy, the original array remains unchanged, and vice versa. This is because a copy owns the data and any changes made to it do not affect the original array. On the other hand, when you make changes to a view, the original array is also modified accordingly. This is because a view does not own the data and any changes made to it affect the original array.

COPY:

Example:

Make a copy, change the original array, and display both arrays:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.copy()
arr[0] = 42

print(arr)
print(x)
```

```
>>[42  2  3  4  5]
[1  2  3  4  5]
```

❖ **Note:** The copy SHOULD NOT be affected by the changes made to the original array.

VIEW:

Example:

Make a view, change the original array, and display both arrays:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
arr[0] = 42

print(arr)
print(x)
```

```
>>[42  2  3  4  5]
[42  2  3  4  5]
```

❖ **Note:** The view SHOULD be affected by the changes made to the original array.

Make Changes in the VIEW:

Example:

Make a view, change the view, and display both arrays:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])
x = arr.view()
x[0] = 31

print(arr)
print(x)
```

```
>>[31  2  3  4  5]
[31  2  3  4  5]
```

Note: The original array SHOULD be affected by the changes made to the view.

Check if Array Owns its Data

As we discussed earlier, when it comes to data ownership, copies own the data, while views don't. However, how can we verify this? Well, every NumPy array comes with an attribute called 'base', which returns 'None' if the array owns the data. In case the array doesn't own the data, the 'base' attribute points to the original object.

Example:

Print the value of the base attribute to check if an array owns its data or not:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5])

x = arr.copy()
y = arr.view()

print(x.base)
print(y.base)
```

```
>>None
[1 2 3 4 5]
```

- ❖ **Note:** The copy returns None.
- ❖ **Note:** The view returns the original array.

Shape of an Array

The shape of a NumPy array is determined by the number of elements in each dimension, which can be accessed using the **'shape'** attribute that returns a tuple.

Example:

Print the shape of a 2-D array:

```
import numpy as np
```

```
arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
```

```
print(arr.shape)
```

```
>>(2, 4)
```

The example above returns (2, 4), which means that the array has 2 dimensions, where the first dimension has 2 elements and the second has 4.

Example:

Create an array with 5 dimensions using `ndmin` using a vector with values 1,2,3,4 and verify that last dimension has value 4:

```
import numpy as np

arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)
print('shape of array :', arr.shape)
```

```
>>[[[[[1 2 3 4]]]]]
shape of array : (1, 1, 1, 1, 4)
```

Reshaping arrays

Reshaping an array means changing its shape by adding or removing dimensions, or changing the number of elements in each dimension.

Reshape From 1-D to 2-D

Example:

Convert the following 1-D array with 12 elements into a 2-D array.

The outermost dimension will have 4 arrays, each with 3 elements:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
```

```
newarr = arr.reshape(4, 3)
```

```
print(newarr)
```

```
>>[[ 1  2  3]
     [ 4  5  6]
     [ 7  8  9]
     [10 11 12]]
```

Reshape From 1-D to 3-D

Example:

Convert the following 1-D array with 12 elements into a 3-D array.

The outermost dimension will have 2 arrays that contains 3 arrays, each with 2 elements:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
```

```
newarr = arr.reshape(2, 3, 2)
```

```
print(newarr)
```

```
>>
[[[ 1  2]
  [ 3  4]
  [ 5  6]]

 [[ 7  8]
  [ 9 10]
  [11 12]]]
```

Can We Reshape Into any Shape?

As long as the number of elements required for reshaping is the same in both shapes, we can reshape an array. For example, an 8-element 1D array can be reshaped into a 4-element 2D array with 2 rows. However, it cannot be reshaped into a 3x3 (9-element) 2D array, as that would require more elements than are available in the original array.

Returns Copy or View?

Example:

Check if the returned array is a copy or a view:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

print(arr.reshape(2, 4).base)

>>[1 2 3 4 5 6 7 8]
```

The example above returns the original array, so it is a view.

Unknown Dimension

It is possible to have one dimension marked as "unknown" in the NumPy reshape method. You can do this by passing -1 as the value for that dimension, and NumPy will automatically calculate the correct number for you. This means that you don't have to specify an exact number for that dimension.

Example:

Convert 1D array with 8 elements to 3D array with 2x2 elements:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])

newarr = arr.reshape(2, 2, -1)

print(newarr)
```

```
>>[[[1 2]
      [3 4]

      [[5 6]
      [7 8]]]
```

❖ **Note:** We can not pass -1 to more than one dimension.

Flattening the arrays

Converting a multi-dimensional array to a 1D array is called flattening. We can use the method `reshape(-1)` to achieve this.

Example:

Convert the array into a 1D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

newarr = arr.reshape(-1)

print(newarr)

>>[1 2 3 4 5 6]
```

- ❖ **Note:** There are a lot of functions for changing the shapes of arrays in NumPy `flatten`, `ravel` and also for rearranging the elements `rot90`, `flip`, `fliplr`, `flipud` etc. These fall under Intermediate to Advanced section of NumPy.

Iterating Arrays

In programming, iterating refers to the process of going through each element in a sequence one by one. When working with multi-dimensional arrays in numpy, we can use a basic for loop in Python to iterate through the elements. If we apply iteration to a one-dimensional array, it will traverse each element in the array individually.

Example:

Iterate on the elements of the following 1-D array:

```
import numpy as np

arr = np.array([1, 2, 3])

for x in arr:
    print(x)
```

```
>>>
1
2
3
```

Iterating 2-D Arrays

In a 2-D array, it will go through all the rows.

Example:

Iterate on the elements of the following 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
    print(x)
```

```
>>
```

```
[1 2 3]
[4 5 6]
```

❖ **Note:** If we iterate on a n-D array it will go through the n-1th dimension one by one.

To return the actual values, the scalars, we have to iterate the arrays in each dimension.

Example:

Iterate on each scalar element of the 2-D array:

```
import numpy as np

arr = np.array([[1, 2, 3], [4, 5, 6]])

for x in arr:
    for y in x:
        print(y)
```

```
>>
1
2
3
4
5
6
```

Iterating Arrays Using `nditer()`

The function `nditer()` is a helping function that can be used from very basic to very advanced iterations. It solves some basic issues which we face in iteration; let us go through it with examples.

Iterating on Each Scalar Element

In basic `for` loops, iterating through each scalar of an array, we need to use `n for` loops, which can be difficult to write for arrays with very high dimensionality.

Example:

Iterate through the following 3-D array:

```
import numpy as np

arr = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])

for x in np.nditer(arr):
    print(x)
```

```
>>
1
2
3
4
5
6
7
8
```

Enumerated Iteration Using ndenumerate()

Enumeration involves listing items in a sequential order. The `ndenumerate()` method can be used to obtain the index of each element during iteration.

Example:

Enumerate on following 2D array's elements:

```
import numpy as np

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

for idx, x in np.ndenumerate(arr):
    print(idx, x)
```

```
>>
(0, 0) 1
(0, 1) 2
(0, 2) 3
(0, 3) 4
(1, 0) 5
(1, 1) 6
(1, 2) 7
(1, 3) 8
```

Joining/Stacking NumPy Arrays

When we combine the contents of two or more arrays into a single array, we call it joining. In SQL, we join tables by keys, but in NumPy, we join arrays by axes. To concatenate arrays, we use the `concatenate()` function and provide a sequence of arrays that we want to join along with the axis we want to use. If we don't pass the axis, the function considers it as 0.

Example:

Join two arrays

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.concatenate((arr1, arr2))

print(arr)
```

```
>>[1 2 3 4 5 6]
```

Example:

Join two 2-D arrays along rows (axis=1):

```
import numpy as np

arr1 = np.array([[1, 2], [3, 4]])

arr2 = np.array([[5, 6], [7, 8]])

arr = np.concatenate((arr1, arr2), axis=1)

print(arr)
```

```
>>[[1 2 5 6]
     [3 4 7 8]]
```

Joining Arrays Using Stack Functions

When it comes to arrays in Python, stacking and concatenation may seem similar, but there is a key difference. Stacking is when two or more arrays are combined along a new axis, whereas concatenation is simply combining two or more arrays end-to-end.

For example, if we have two 1-D arrays, we can stack them along the second axis which would result in them being put on top of one another.

To perform stacking in Python, we can use the `stack()` method. This method takes a sequence of arrays that we want to join, along with the axis. If the axis is not specified, it is taken as 0.

Example:

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.stack((arr1, arr2), axis=1)

print(arr)
```

```
>>[[1 4]
    [2 5]
    [3 6]]
```

Stacking Along Rows

NumPy provides a helper function, `hstack()`, to stack along rows.

Example:

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.hstack((arr1, arr2))
print(arr)
```

```
>>[1 2 3 4 5 6]
```

Stacking Along Columns

NumPy provides a helper function, `vstack()`, to stack along columns.

Example:

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.vstack((arr1, arr2))
print(arr)
```

```
>>[[1 2 3]
    [4 5 6]]
```

Stacking Along Height (depth)

NumPy provides a helper function, `dstack()`, to stack along height, which is the same as depth.

Example:

```
import numpy as np

arr1 = np.array([1, 2, 3])

arr2 = np.array([4, 5, 6])

arr = np.dstack ((arr1, arr2))
print(arr)
```

```
>>[[[1 4]
      [2 5]
      [3 6]]]
```

Splitting NumPy Arrays

To put it simply, Splitting is the opposite of Joining.

When we Join, we combine several arrays into one, but when we Split, we divide one array into multiple. To Split an array, we use the `array_split()` method and pass in the array we want to split along with the number of splits we want to create.

Example:

Split the array in 3 parts:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6])
```

```
newarr = np.array_split(arr, 3)
```

```
print(newarr)
```

```
>>[array([1, 2]), array([3, 4]), array([5, 6])]
```

❖ **Note:** The return value is a list containing three arrays.

If the array has less elements than required, it will adjust from the end accordingly.

Example:

Split the array in 4 parts:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6])
```

```
newarr = np.array_split(arr, 4)
```

```
print(newarr)
```

```
>>[array([1, 2]), array([3, 4]), array([5]), array([6])]
```

- ❖ **Note:** We also have the method `split()` available but it will not adjust the elements when elements are less in source array for splitting like in example above, `array_split()` worked properly but `split()` would fail.

Searching Arrays

You can use the `where()` method to search an array and return matching indexes.

Example:

Find the indexes where the value is 4:

```
import numpy as np

arr = np.array([1, 2, 3, 4, 5, 4, 4])

x = np.where(arr == 4)

print(x)

>>(array([3, 5, 6]),)
```

The example above will return a tuple: `(array([3, 5, 6]),)`

This means that the value 4 is present at index 3, 5, and 6.

Example:

Find the indexes where the values are odd:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
x = np.where(arr%2 == 1)
```

```
print(x)
```

```
>>(array([0, 2, 4, 6], dtype=int64),)
```

Example:

Find the indexes where the values are even:

```
import numpy as np
```

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
x = np.where(arr%2 == 0)
```

```
print(x)
```

```
>>(array([1, 3, 5, 7], dtype=int64),)
```

Search Sorted

The `searchsorted()` method performs a binary search in a sorted array and returns the index where the specified value can be inserted while maintaining the search order. It is important to note that this method is designed to be used specifically on sorted arrays.

Example:

Find the indexes where the value 7 should be inserted:

```
import numpy as np

arr = np.array([6, 7, 8, 9])

x = np.searchsorted(arr, 7)

print(x)

>>1
```

Example explained: The number 7 should be inserted on index 1 to remain the sort order.

The method starts the search from the left and returns the first index where the number 7 is no longer larger than the next value.

Multiple Values Search

To search for more than one value, use an array with the specified values.

Example:

Find the indexes where the values 2, 4, and 6 should be inserted:

```
import numpy as np

arr = np.array([1, 3, 5, 7])

x = np.searchsorted(arr, [2, 4, 6])

print(x)

>>[1 2 3]
```

The return value is an array: `[1 2 3]` containing the three indexes where 2, 4, 6 would be inserted in the original array to maintain the order.

Sorting Arrays

Sorting refers to the process of arranging elements in a specific order, typically in an ascending or descending numerical or alphabetical sequence. The NumPy ndarray object provides a built-in function known as 'sort()', which can be used to sort a specified array. An ordered sequence is a sequence that has a specific order that corresponds to the elements present in it.

Example:

Sort the array:

```
import numpy as np

arr = np.array([3, 2, 0, 1])

print(np.sort(arr))

>>[0 1 2 3]
```

❖ **Note:** This method returns a copy of the array, leaving the original array unchanged.

Filtering Arrays

In NumPy, you can filter an array using a boolean index list. This boolean index list is a list of booleans that relates to the indexes in the original array.

If the value at a certain index is True, the element at that index will be included in the filtered array. Conversely, if the value at that index is False, the element will be excluded from the filtered array.

Example:

Create an array from the elements on index 0 and 2:

```
import numpy as np

arr = np.array([41, 42, 43, 44])

x = [True, False, True, False]

newarr = arr[x]

print(newarr)

>>[41 43]
```

given code generates a new array that includes only the values where the filter array has a value of True. In the example provided, the new array includes only the values at index 0 and 2 of the original array, which are 41 and 43 respectively.

Arithmetic operations

All the usual arithmetic operators (+, -, *, /, //, **, etc.) can be used with `ndarrays`. They apply elementwise:

Example:

Create an array from the elements on index 0 and 2:

```
import numpy as np

a = np.array([14, 23, 32, 41])
b = np.array([5, 4, 3, 2])

print("a + b =", a + b)
print("a - b =", a - b)
print("a * b =", a * b)
print("a / b =", a / b)
print("a // b =", a // b)
print("a % b =", a % b)
print("a ** b =", a ** b)
```

```
>>a + b = [19 27 35 43]
a - b = [ 9 19 29 39]
a * b = [70 92 96 82]
a / b = [ 2.8      5.75      10.66666667 20.5      ]
a // b = [ 2  5 10 20]
a % b = [4 3 2 1]
a ** b = [537824 279841 32768 1681]
```

❖ **Note:** The multiplication is not a matrix multiplication. The arrays must have the same shape. If they do not, NumPy will return a `broadcasting` error.

Thank you